

ScholarPath — Class Diagrams

UML Object Model & Architecture

ScholarPath Team

Abstract

This is Part 4 — the final instalment — of a four-part teaching series that walks the ScholarPath data model from a plain Entity–Relationship Diagram (ERD) through an Enhanced ER (EERD), then a Relational Mapping, and finally to the object-oriented design presented here. Where Part 3 produced normalised *tables*, this document presents the *classes* that those tables become at run time: the C# domain entities of the ScholarPath back-end. We explain why a UML class diagram is the right lens for an object-oriented codebase, fix a precise notation legend, establish the inheritance foundation (`BaseEntity` → `AuditableEntity`, the `ISoftDeletable` interface, and the special case of `ApplicationUser`), then tour the domain cluster by cluster. We close on the architectural class view — the Clean Architecture *ports and adapters* seam — which shows how the Application layer depends only on interfaces while the Infrastructure layer supplies swappable implementations. Every entity, attribute, and association named here is grounded in the actual source tree (`server/src/ScholarPath.Domain/Entities/*` and `ScholarPath.Application/Common/Interfaces/*`).

Contents

1	From the Relational Schema to the Object Model	1
1.1	Why a class diagram?	1
1.2	UML notation legend	1
2	The Domain Foundation: Inheritance and the Soft-Delete Contract	2
2.1	<code>BaseEntity</code> → <code>AuditableEntity</code>	2
2.2	The <code>ISoftDeletable</code> interface	2
2.3	The special case: <code>ApplicationUser</code> extends ASP.NET Identity	3
3	The Domain Model, Cluster by Cluster	3
3.1	Base types	4
3.2	Identity & Profile	5
3.3	Scholarships & Applications	6
3.4	Booking, Payments & Ratings	7
3.5	Community & Chat	8
3.6	Resources & Notifications	8
3.7	AI Platform	9
4	Clean Architecture: The Ports-and-Adapters Seam	9

4.1	Ports defined by the Application layer	9
4.2	Adapters provided by the Infrastructure layer	9
5	Conclusion	11

1 From the Relational Schema to the Object Model

1.1 Why a class diagram?

Parts 1–3 of this series modelled ScholarPath as *data*: entities, relationships, and finally relational tables with primary and foreign keys. That view is correct, but it is incomplete for a system that is *written* in an object-oriented language. The ScholarPath back-end is built on .NET 10 / C#, where the persistent data does not live as bare rows; it lives as *objects* — instances of domain classes that carry both state (attributes) and behaviour (operations). Entity Framework Core then maps those classes onto the relational tables produced in Part 3. The class diagram is the bridge: it is the design artefact that the code actually realises.

A UML class diagram answers questions the ERD cannot. It distinguishes *inheritance* (an `AuditableEntity` *is a* `BaseEntity`) from *association* (a `ConsultantBooking` *references* an `ApplicationUser`). It distinguishes a *whole/part* relationship where the part dies with the whole (composition) from one where the part lives independently (aggregation). And it lets us model *interfaces* — contracts with no data of their own — which have no equivalent in a pure relational schema but are central to how ScholarPath stays testable and provider-agnostic. The diagrams in this document are authored in the style of *Sommerville, Software Engineering (9th ed.)* and rendered both as Mermaid `classDiagram` views and as vector PlantUML PDFs.

1.2 UML notation legend

The following notation is used consistently across every figure in this document. Reading it once here makes the rest of the diagrams self-explanatory.

Table 1: UML class-diagram notation used throughout this document.

UML element	Glyph	Meaning in ScholarPath
Generalization	Base < -- Derived	Inheritance: the derived class <i>is a</i> kind of the base (e.g. <code>AuditableEntity</code> ← <code>Scholarship</code>). Hollow triangle points at the parent.
Realization	Iface < .. Class	A class <i>implements</i> an interface (port). Dashed line, hollow triangle (e.g. <code>StripeService</code> realizes <code>IStripeService</code>).
Composition	Whole *- Part	The part cannot exist without the whole; filled diamond at the whole. Deleting the whole deletes the parts (e.g. <code>ChatConversation</code> *- <code>ChatMessage</code>).
Aggregation	Whole o- Part	A weaker whole/part link; the part can outlive the whole. Hollow diamond (e.g. <code>Payout</code> o- <code>Payment</code>).
Association	A "1" -> "*" B	A plain reference with role names and multiplicities at each end (one-to-one, one-to-many, many-to-many).
Stereotype	«abstract», «interface»	Marks an abstract base class, an interface (port), or a junction table.

Multiplicities are read at the far end of the line: A "1" -> "*" B means one A is associated with many B, and a 0..1 end denotes an optional reference.

2 The Domain Foundation: Inheritance and the Soft-Delete Contract

Before the individual entities make sense, three foundational types must be understood, because almost every other class is defined in terms of them.

2.1 `BaseEntity` → `AuditableEntity`

`BaseEntity` is the abstract root of the domain. It carries the identity of every persistent object — a `Guid Id` — and the domain-event plumbing: a read-only `DomainEvents` collection plus `RaiseDomainEvent(e)` and `ClearDomainEvents()` operations. This lets an entity record that something significant happened (for example, a booking was confirmed) without knowing who will handle it; MediatR dispatches those events after the unit of work commits.

`AuditableEntity` is an abstract class that generalizes `BaseEntity` and adds the audit and concurrency columns that Part 3 showed on nearly every table: `CreatedAt`, `CreatedByUserId`, `UpdatedAt`, `UpdatedByUserId`, and a `RowVersion` optimistic-concurrency token. The overwhelming majority of ScholarPath’s business entities — `Scholarship`, `ApplicationTracker`, `UserProfile`, `Document`, `ConsultantBooking`, `Payment`, and so on — generalize `AuditableEntity`, which is exactly why the relational mapping in Part 3 repeated those five audit columns on table after table. Inheritance in the object model explains the apparent duplication in the schema.

2.2 The `ISoftDeletable` interface

`ISoftDeletable` is an «interface»: a behavioural contract with no data of its own that declares the soft-delete state — `IsDeleted`, `DeletedAt`, and `DeletedByUserId`. Entities that realize it are never physically removed; instead a global EF Core query filter hides them. This is the object-model counterpart of the soft-delete columns and filtered indexes discussed in Part 3. The class diagrams draw the realization on a representative subset for legibility, but the contract is realized far more widely — in addition to `ApplicationUser`, `Scholarship`, and `ApplicationTracker`, it is realized by `Document`, `CompanyReview` and `CompanyReviewRequest`, `ConsultantReview`, `ConsultantAvailability`, `ConsultantBooking`, `Payment`, `ChatMessage`, `ForumPost`, `Resource`, `Notification`, `SessionRecording`, and `SuccessStory`, among others.

2.3 The special case: `ApplicationUser` extends ASP.NET Identity

There is one deliberate and important exception to the inheritance hierarchy. `ApplicationUser` does **not** extend `BaseEntity` or `AuditableEntity`. It extends ASP.NET Core Identity’s `IdentityUser<Guid>`, and `ApplicationRole` extends `IdentityRole<Guid>`. This is required so that the framework’s authentication, password-hashing, lockout, and role machinery operate on the user class directly. The trade-off is that, because C# is single-inheritance, `ApplicationUser` cannot also inherit `BaseEntity`; it therefore re-implements the domain-event plumbing manually and realizes `ISoftDeletable` on its own. Beyond the Identity members it adds the ScholarPath-specific fields `Email`, `FirstName`, `LastName`, an `AccountStatus`, the multi-role `ActiveRole` selector, `IsOnboardingComplete`, and a `FullName()` operation. Recognising this exception is essential when reading the schema: the `AspNetUsers` table is shaped by Identity, not by the ScholarPath base classes.

3 The Domain Model, Cluster by Cluster

Figure 1 presents the full domain model on one canvas — every business entity, its generalization to the base types, and the principal associations between clusters. The subsections that follow zoom into each cluster so the multiplicities and the composition-versus-aggregation choices can be read clearly.

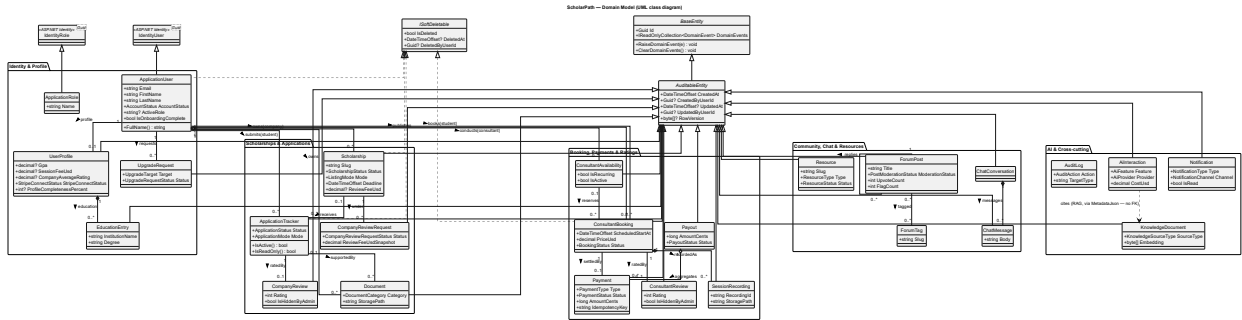


Figure 1: ScholarPath full UML domain model: base types, identity, and every business cluster with their associations.

3.1 Base types

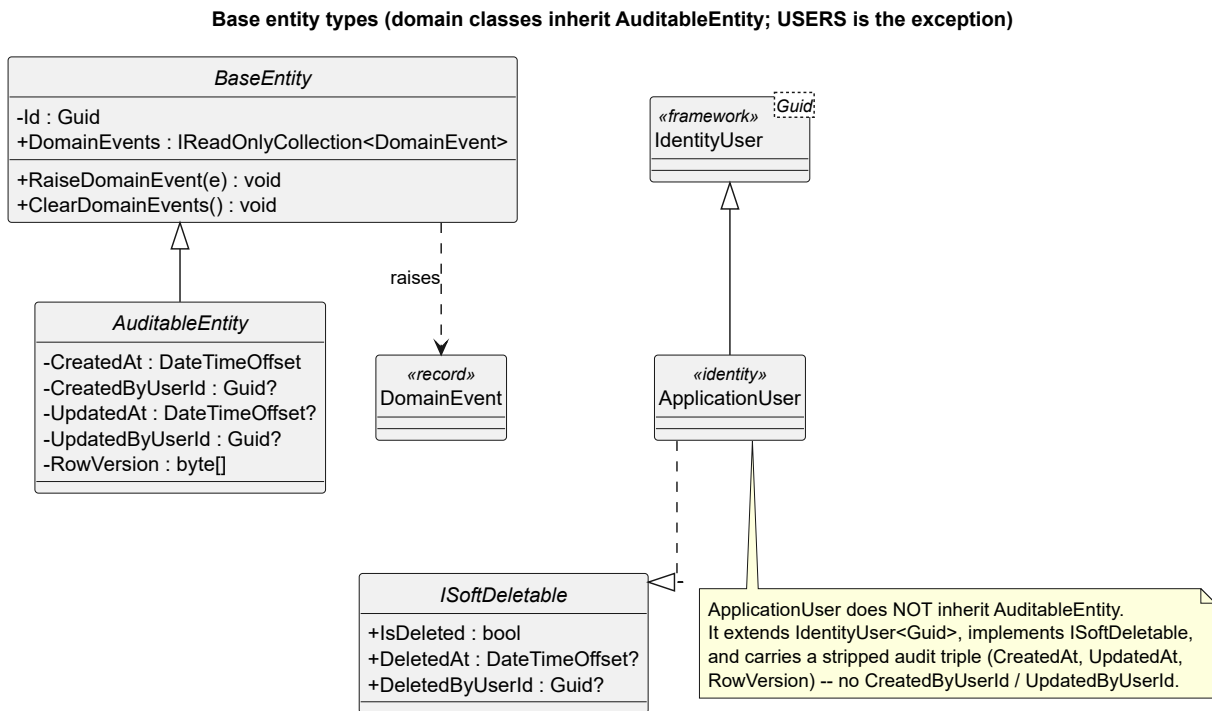


Figure 2: The inheritance foundation: **BaseEntity** → **AuditableEntity**, the **ISoftDeletable** interface, and the ASP.NET Identity bases.

This figure isolates the foundation described in Section 2. It shows **BaseEntity** as the abstract root, **AuditableEntity** generalizing it with the audit/concurrency fields, the **ISoftDeletable** interface, and the two ASP.NET Identity base classes that **ApplicationUser** and **ApplicationRole** extend. Every subsequent cluster figure assumes these parents and omits redundant audit columns to keep each diagram readable.

3.2 Identity & Profile

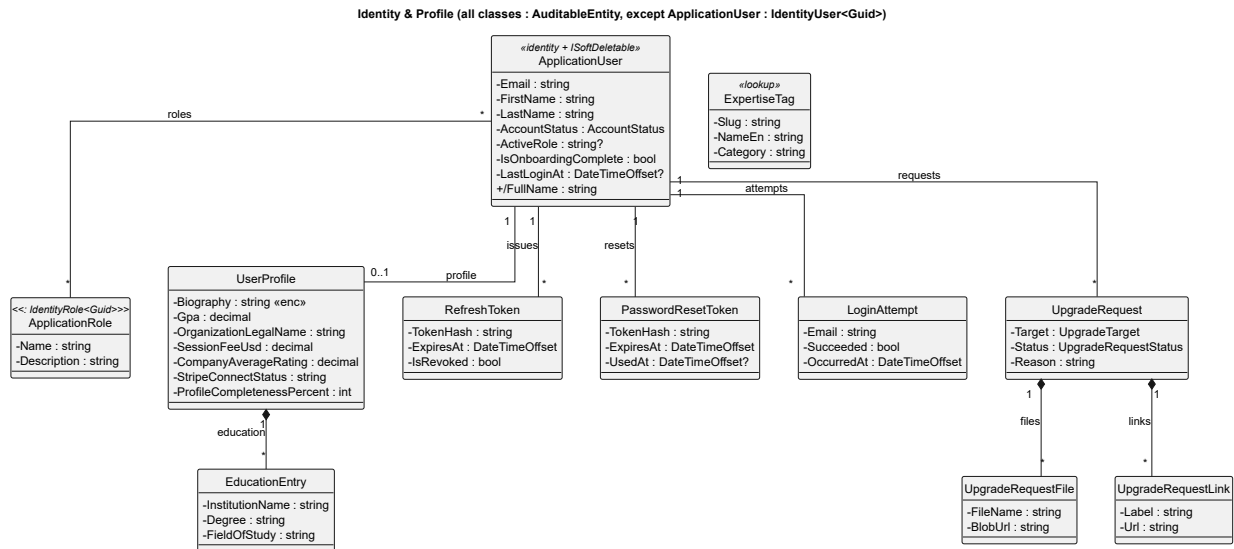


Figure 3: Identity and profile cluster: ApplicationUser, ApplicationRole, UserProfile, and EducationEntry.

An ApplicationUser has at most one UserProfile (1 → 0..1), which holds the rich, role-specific attributes: Gpa, SessionFeeUsd, CompanyAverageRating, the StripeConnectStatus, and a computed ProfileCompletenessPercent. The relationship between UserProfile and EducationEntry is a **composition** (UserProfile *— EducationEntry): an education record describes the academic history of exactly one profile and has no independent meaning, so it is owned by — and deleted with — its profile.

3.3 Scholarships & Applications

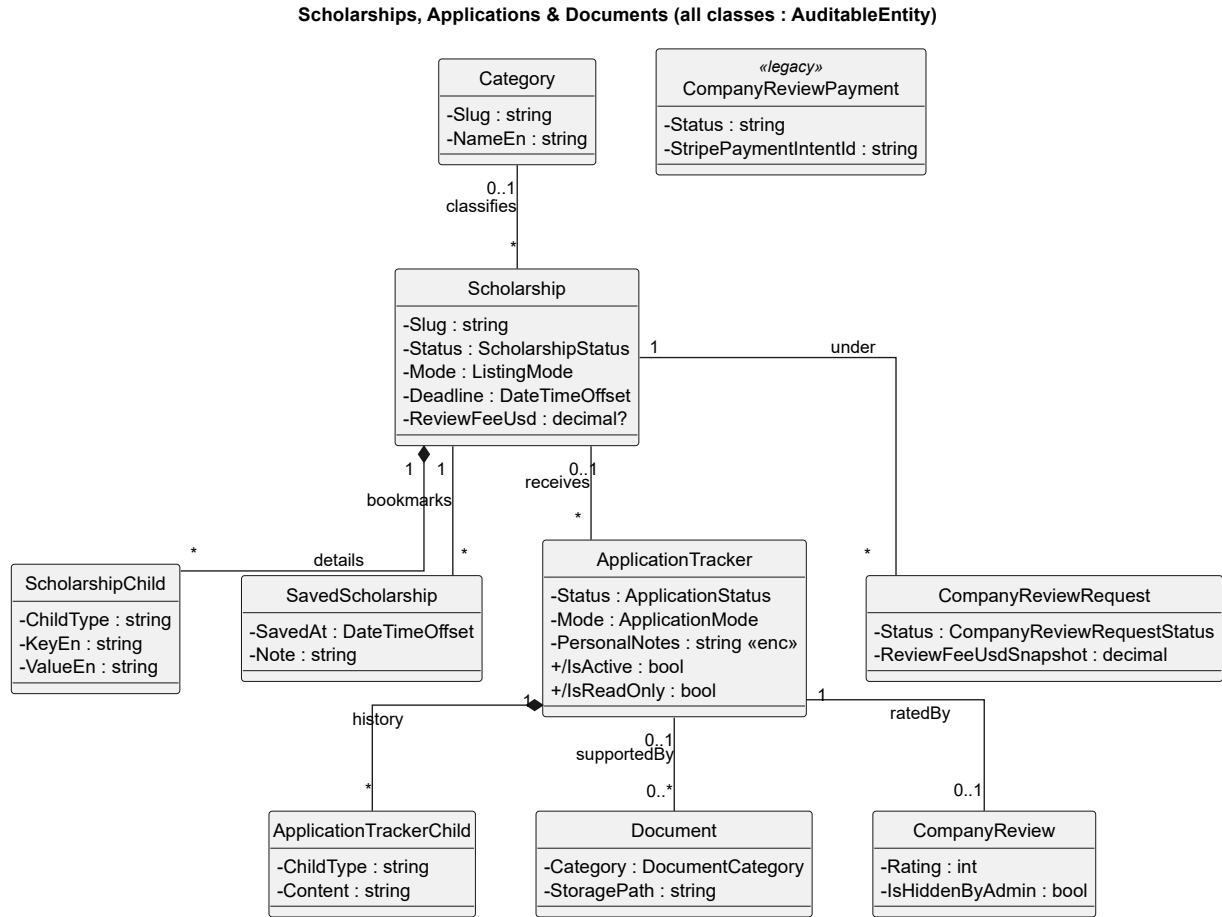


Figure 4: Scholarships and applications cluster: Scholarship, ApplicationTracker, Document, and the company-review path.

A **Scholarship** (with its **Slug**, **Status**, listing **Mode**, **Deadline**, and optional **ReviewFeeUsd**) is owned by at most one company user — modelled as an *aggregation* (**ApplicationUser** o-**Scholarship**) because a scholarship can outlive a change of ownership and is not destroyed merely because the listing is re-assigned. A student submits many **ApplicationTracker** rows, each carrying an **ApplicationStatus**, an **ApplicationMode**, a **SubmittedAt** timestamp, and behaviour such as **IsActive()** and **IsReadOnly()** that governs the state machine.

A point worth emphasising for the committee: **Document** is associated with an **ApplicationTracker** as **one-to-many** (**ApplicationTracker** "0..1" -> "*" **Document**), *not* as an optional single attachment. One application is supported by many documents (transcript, recommendation letters, essays, and so on), and each document is also owned by the uploading **ApplicationUser**. The company-review path adds **CompanyReviewRequest** (raised by a student against a scholarship, with a **ReviewFeeUsdSnapshot**) and the resulting **CompanyReview** rating that an **ApplicationTracker** may receive.

3.4 Booking, Payments & Ratings

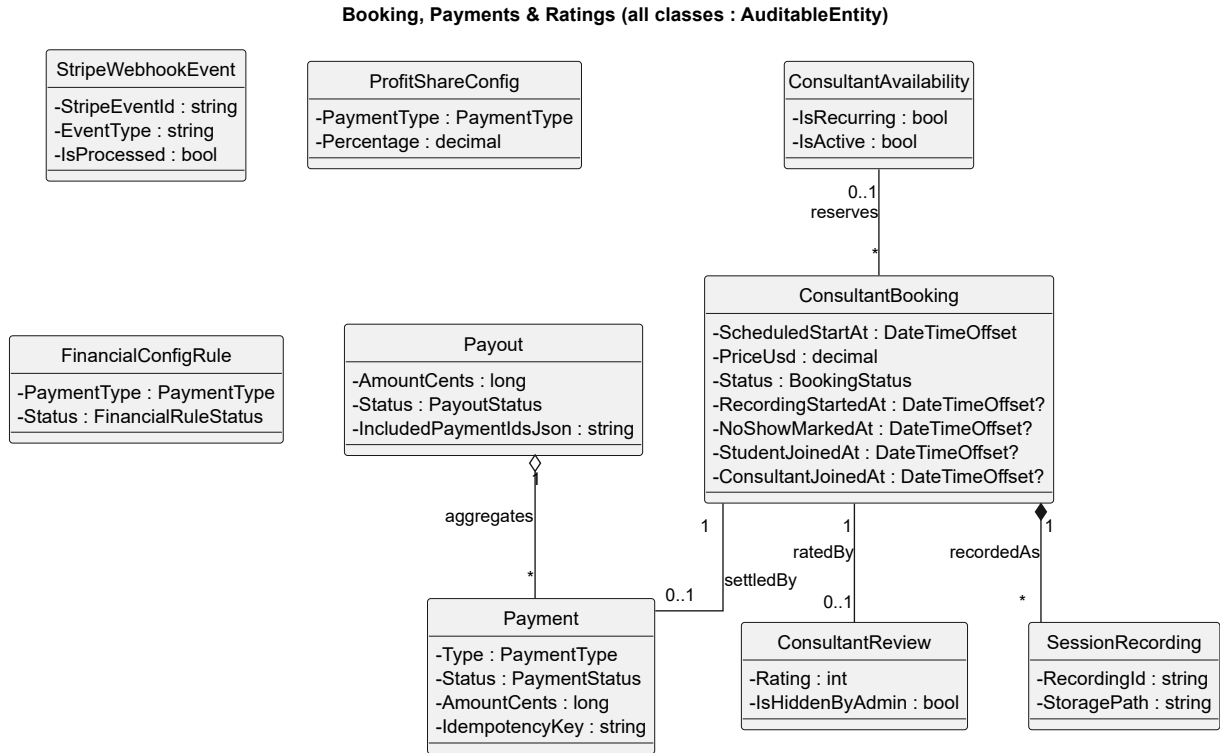


Figure 5: Consultant booking, payments, payouts, ratings, and session recordings.

A consultant publishes **ConsultantAvailability** slots (aggregation: a slot can be deactivated yet retained for history), which a student reserves as a **ConsultantBooking** carrying **ScheduledStartAt**, **DurationMinutes**, **PriceUsd**, a **BookingStatus**, and a no-show flag. The same user class appears at both ends of the booking — once as the booking student, once as the conducting consultant — which is why role names on the associations matter. A booking is settled by at most one **Payment** and rated by at most one **ConsultantReview**.

Two modelling choices are notable. First, **ConsultantBooking** *— **SessionRecording** is a **composition**: a recording has no life outside the booking that produced it, so it is destroyed with the booking. Second, **Payout** o— **Payment** is an **aggregation**: a payout batches many already-existing payments and the payments survive the payout record. As Part 3 noted, the **Payment/Payout** links are drawn as UML associations but are *application-enforced* rather than backed by database foreign keys. The **Payment** class also carries the money-split fields (**AmountCents**, **ProfitShareAmountCents**, **PayeeAmountCents**) and an **IdempotencyKey** that makes retries safe.

3.5 Community & Chat

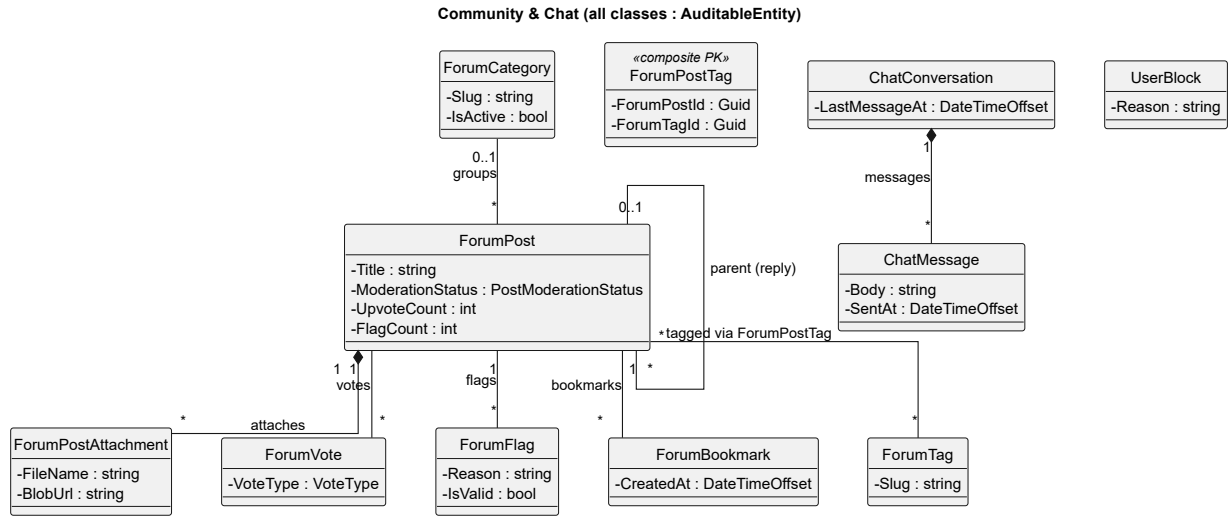


Figure 6: Community forum (posts, votes, flags, tags) and direct chat (conversations and messages).

A `ForumPost` carries a self-association to model threaded replies (`ForumPost` "1" -> "0..1" `ForumPost` : `parent`). Its `ForumVote` and `ForumFlag` children are **compositions** — a vote or a flag is meaningless without the post it targets. Tagging is a many-to-many resolved through the `ForumPostTag` «junction» class, mirroring the bridge table from Part 3. On the chat side, a `ChatConversation` is a **composition** of `ChatMessage` objects (`ChatConversation` *- `ChatMessage`): messages belong wholly to their conversation and are removed with it.

3.6 Resources & Notifications

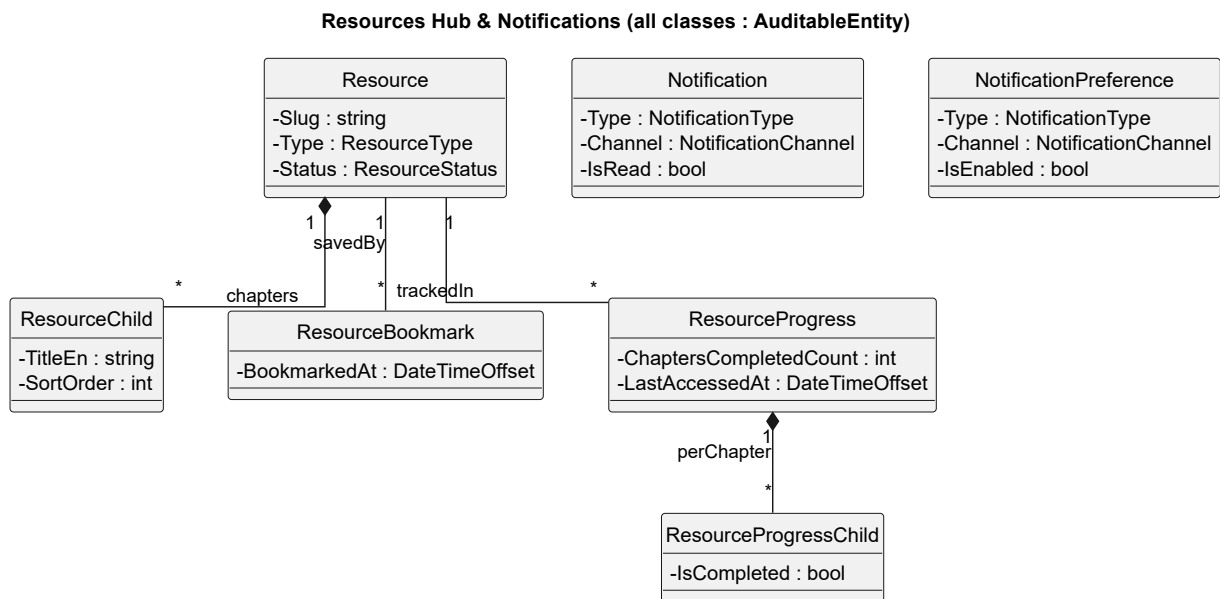


Figure 7: Learning resources (with their chapters and per-user progress) and the notification entity.

A `Resource` (Slug, Type, Status, AuthorRole) is composed of ordered `ResourceChild` chapters (`Resource` *- `ResourceChild`) and is tracked by many `ResourceProgress` rows, one per

learner, recording `ChaptersCompletedCount`. `Notification` captures a typed, multi-channel alert (`NotificationType`, `NotificationChannel`, `IsRead`) and is the entity written by the notification dispatcher described in Section 4.

3.7 AI Platform

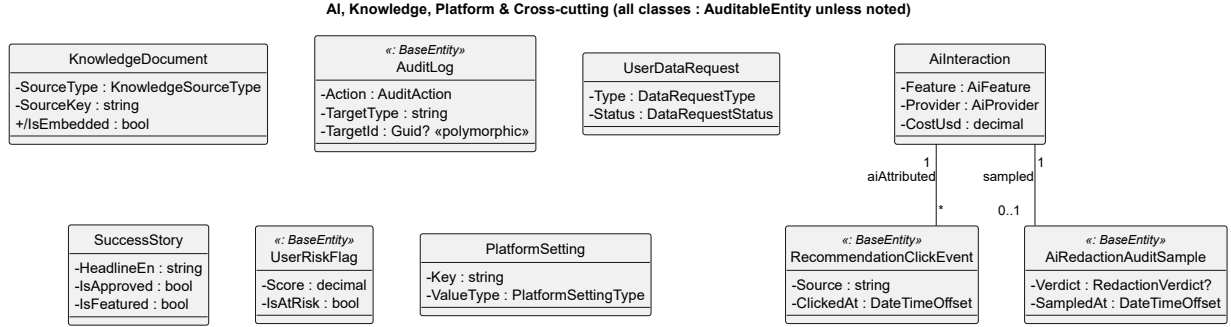


Figure 8: AI platform entities: `AiInteraction` usage records and the `KnowledgeDocument` RAG corpus.

`AiInteraction` logs every model call — the `AiFeature` invoked, the `AiProvider` used, the `CostUsd`, and token counts such as `PromptTokens` — giving the platform full cost and audit visibility. `KnowledgeDocument` is the Retrieval-Augmented Generation (RAG) corpus, storing a `SourceType`, a vector `Embedding`, and a `ContentHash` for deduplication. The link `AiInteraction ..> KnowledgeDocument` is a *dependency*, not an association with a foreign key: an interaction cites the documents it retrieved through a `MetadataJson` field rather than a relational reference, which is why Part 3’s schema shows no FK between them.

4 Clean Architecture: The Ports-and-Adapters Seam

4.1 Ports defined by the Application layer

ScholarPath follows Clean Architecture: dependencies point inward (`API` → `Application` → `Domain`; `Infrastructure` → `Application/Domain`). The crucial design device is the *port*: the Application layer declares a set of interfaces describing *what* it needs, with no knowledge of *how* those needs are met. These ports include `IAiService`, `IEmbeddingService`, `IStripeService`, `IMeetingService`, `IEmailService`, `IBlobStorageService`, `IFileScanService`, `IFieldEncryptionService`, `IPowerBiService`, `IEventPublisher`, `INotificationDispatcher`, the realtime notifiers (`IChatRealtimeNotifier`, `ICommunityRealtimeNotifier`), and the persistence port `IApplicationDbContext`. Because use-cases depend only on these abstractions, the business logic is fully unit-testable with in-memory fakes.

4.2 Adapters provided by the Infrastructure layer

Figure 9 shows the seam: each port (interface) on the left is *realized* by one or more adapters on the right, and the concrete adapter is chosen at start-up by dependency injection and configuration (for example `Ai__Provider` and the hosting environment). This is what lets ScholarPath run fully offline in development yet hit real Azure services in production without changing a single line of application code.

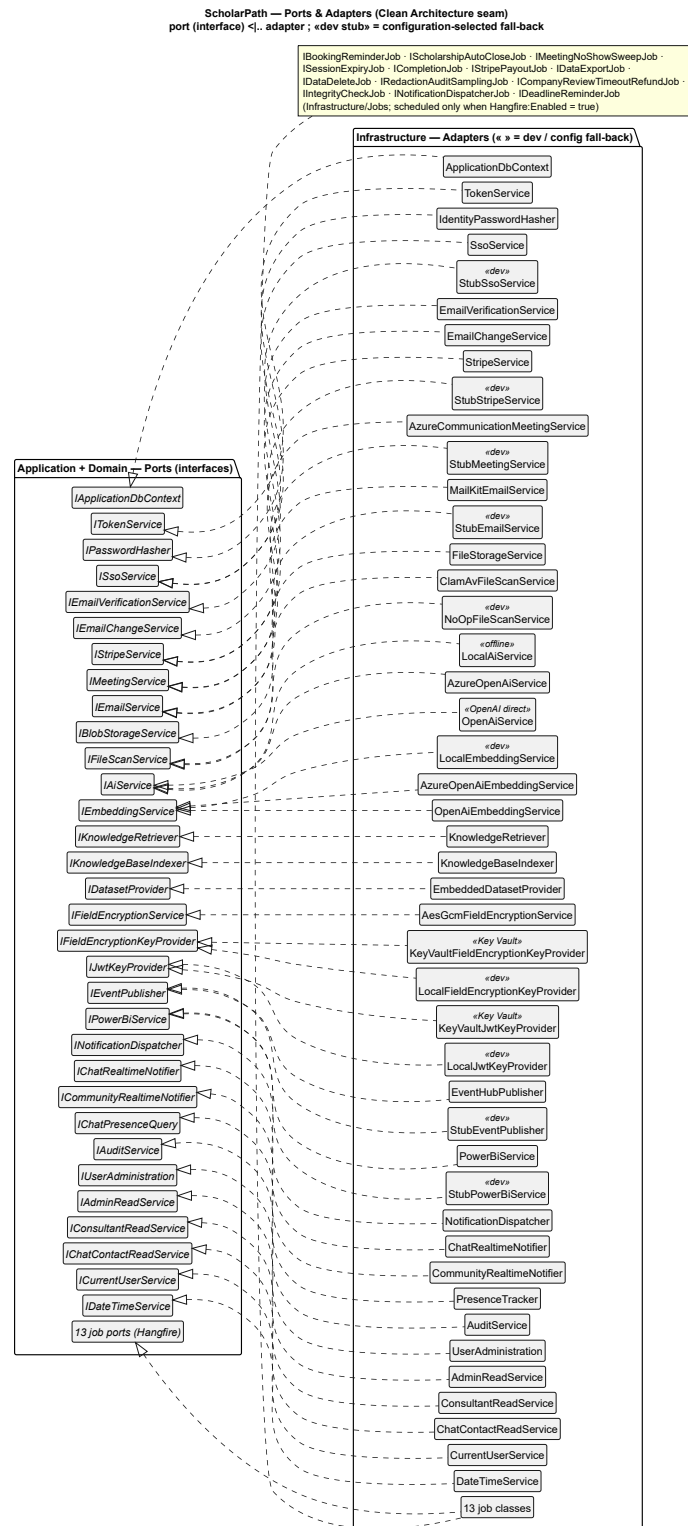


Figure 9: Ports (Application interfaces) realized by Infrastructure adapters, selected by configuration; every port has a production adapter and a dev stub/local fallback.

Table 2 lists the principal port-to-adapter realizations. Notably, the AI port has three adapters — two production providers and one local fallback — selected purely by configuration.

Table 2: Ports and their realizing adapters (production and dev fallback).

Port (Application)	Production adapter	Dev fallback
IApplicationDbContext	ApplicationDbContext (EF Core)	—
IAiService	AzureOpenAiService / OpenAiService	LocalAiService
IEmbeddingService	AzureOpenAiEmbeddingService / OpenAiEmbeddingService	LocalEmbeddingService
IStripeService	StripeService	StubStripeService
IMeetingService	AzureCommunicationMeetingService	StubMeetingService
IService	MailKitEmailService	StubEmailService
IBlobStorageService	FileStorageService (Azure Blob)	Local storage provider
IFileScanService	ClamAvFileScanService	NoOpFileScanService
IFieldEncryptionService	AesGcmFieldEncryptionService + Key Vault	—
IPowerBiService	PowerBiService	StubPowerBiService
IEventPublisher	EventHubPublisher	StubEventPublisher
INotificationDispatcher	NotificationDispatcher	—

The realization relationship in UML (Interface <|.. Class) is exactly the right glyph for this seam. For instance:

Listing 1: Realization of the AI port by three interchangeable adapters.

IAiService	< .. AzureOpenAiService	// Ai__Provider=AzureOpenAi
IAiService	< .. OpenAiService	// Ai__Provider=OpenAi
IAiService	< .. LocalAiService	// dev / offline fallback
IEmbeddingService	< .. AzureOpenAiEmbeddingService	
IEmbeddingService	< .. OpenAiEmbeddingService	
IEmbeddingService	< .. LocalEmbeddingService	
IStripeService	< .. StripeService	
IMeetingService	< .. AzureCommunicationMeetingService	

The production adapters integrate Azure OpenAI (chat + embeddings, with an OpenAI-direct alternative), Stripe (PaymentIntents, capture, refund, Connect payouts), Azure Communication Services (video rooms and recording), MailKit over SMTP, ClamAV for malware scanning of uploads, AES-GCM field encryption keyed by Azure Key Vault, Azure Event Hub feeding Power BI, and the NotificationDispatcher that fans an alert out to the Notifications table, e-mail, and SignalR simultaneously. Every one of these has a dev-time stub or local fallback so the platform behaves identically offline. The wider deployment topology of these integrations is covered in the companion component and deployment diagram.

5 Conclusion

This object model is the executable form of the relational schema produced in Part 3. The tables of the Relational Mapping document become the C# domain classes presented here; the foreign keys become typed associations with explicit multiplicities; the repeated audit columns are explained by the BaseEntity → AuditableEntity inheritance; the soft-delete columns are the realization of ISoftDeletable; and the bridge tables become «junction» classes. Two design decisions deserve a final note for the committee: ApplicationUser extends ASP.NET IdentityUser<Guid> rather than the domain base, and Document is a one-to-many child of ApplicationTracker rather than an optional single attachment. Layered on top, the ports-and-adapters seam keeps the entire domain and application logic independent of any external vendor, which is the property that makes ScholarPath both testable and portable. Together with the ERD, EERD, and Relational Mapping documents that precede it, this completes a single unbroken line from conceptual model to running object-oriented system.